

Group 3:

Luke Hagan (od8653)

Ruba Hagog (je8610)

Manuel Mendoza (ti7022)

Trung Lam (yg3892)

Lemarion Hall (qa1424)

CS 401

Uno Application Report

The first step in our project, after landing on an UNO application, was to draft up a list of trustable requirements in order to give the team structure and direction. The requirements that were decided on were then split into two sections: Function and Non-functional Requirements. Those requirements will then be later used to define how successful the implementation of the application will be. The requirements were as follows:

Functional Requirements:

- The game will be strictly a two player UNO game.
- The game will have only one 'real' player and the other player will be a pre-programmed 'bot'.
- The game will have a randomized stack of cards that the players can draw from.
- The game will have a stack of cards that players can stack on to. At the beginning of the game the stack will have one card placed on it to start the game.
- User is able to log in with a username and password
- Users can start a new game.
- Users can view their User ID.
- Users can view the cards that have been placed on the stack last.
- Users can see their cards but not the other user's cards.
- Users can select from their cards one card to put down when it is their turn.
- Users can see if it's their turn.
- Users can call UNO.
- The game will automatically add two cards to players that receive a "draw 2" card.
- The game will automatically skip players who receive a "skip player" card.
- The game will automatically give the user who put down the card their turn and skip the other user's turn.

- The game will automatically end when either the user or the bot run out of cards.

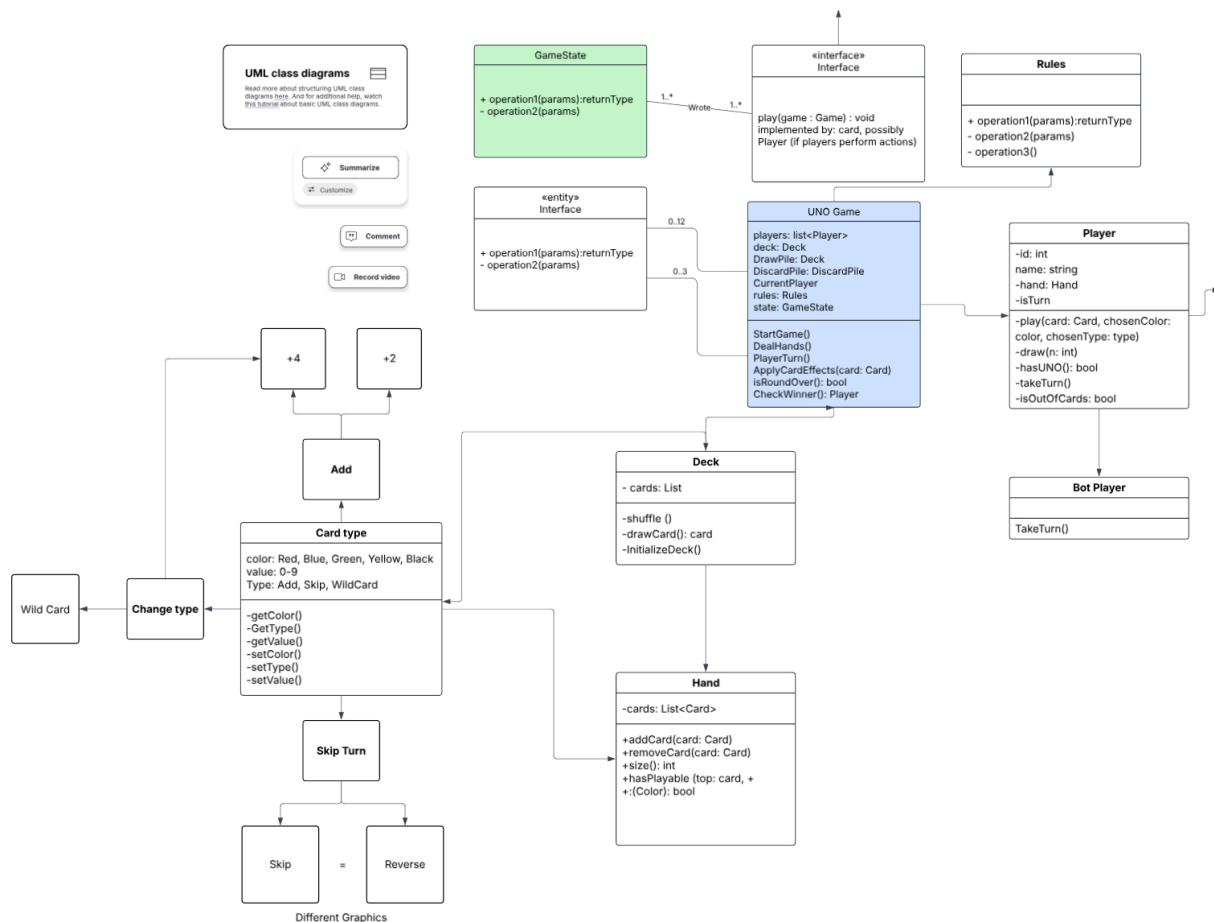
Non- Functional Requirements:

- The UI should be intuitive, requiring no instructions for core mechanics.
- The system should launch and load game sessions within 3-5 seconds.
- The game state (card throws, deck draws) must sync within 100-200 milliseconds to ensure smooth, real-time play.

After identifying the requirements, the team sat down to track the correct flow of the application in order to narrow down how the design of the application will look like, this manifested in the team using a sequence diagram in order to portray that flow. The sequence diagram is as follows:



Then, the next step was to assess the requirements, as well as the sequence diagram, in order to extract some classes to start the implementation of the application.. These classes included: Game, User, Bot, Card and Deck. Then together, a UML Class Diagram was created, as a team, in order to give the software engineering team a map and some direction to follow. The UML Class diagram was as follows:



using these diagrams, the software team can then implement the classes listed above. All of the implementations of the classes were completed by the software engineering team in Java As the software engineering team was working on the backend part of the project, the UI/UX team worked on creating an user interface, so that when the time comes, the team would integrate the User Interface component with the backend component of the application. The UX/UI team used p5, which is a javascript type language in order to implement the user interface.

GitHub and Collaboration:

In order for the team to collaborate effectively, a GitHub repository was created where all group work is concentrated. Later on, as the project progressed, the Project Manager implemented a system of weekly progress check-ins with the team to ensure that, one: everyone is going at the correct pace to meet deadlines, and two: that the team has a deeper understanding of what all of the work that each person/group is doing. This then allowed the team to progress at the correct pace while creating a solid foundation for when the Frontend and the Backend need to intersect.

Repo Link: <https://github.com/lukehag/Group3-UNO>

Class Implementation:

The implementation of the UNO application began with translating the UML class diagram into concrete Java classes. The core classes defined included Card, Deck, Hand, Player, Opponent, and UnoGame, each encapsulating a specific responsibility within the game system. The Card class was implemented with Color and Type enumerations to represent the full range of UNO cards, along with methods such as `isMatching()` to handle the logic behind whether or not a card can be played. The Deck class was responsible for any interactions involving the draw pile such as creating, shuffling, and distributing cards. While the Hand class managed the array that represents each player's current set of cards. The Player class defined the core behavior shared by all players, and the Opponent class extended it with automated decision-making logic such as `findBestCard()` and `mostCommonColor()` to simulate an AI opponent. The AI prioritizes playing action cards with the same color as the top card then it tries to match the color and number. Finally it will play a Draw 4 or Wild card when it has no other option, otherwise it will just draw a card then skip.

Before integrating a graphical interface, the application was first implemented and tested as a text-based console program. This approach allowed the team to verify that the core game logic was functioning correctly in isolation, without the added complexity of a user interface. By running the game through the terminal, it was straightforward to trace the flow of each turn, inspect card values, confirm that special card effects such as Skip, Reverse, Draw Two, and Wild were being applied correctly, and validate win conditions. This text-based testing phase was essential for identifying and resolving logic errors early — such as ensuring the correct player was affected by opener cards, that the draw pile was properly reshuffled when empty, and that

the turn order advanced correctly under all conditions. Once the class implementations were verified through console testing, the application was then extended with the REST API layer and graphical interface.

System Integration:

After the UX/UI, as well as the software team were done with their respective implementations, it was time to integrate the two parties. Of course the Frontend programming would need some adjustments in order to fit the finalized version of the backend code. The way these two would connect is through the REST API where the java backend exposes endpoints that return data in JSON format. This would then allow you to create a `fetch()` API or other libraries like Axios to send requests to the java server, that is when we introduced the **UnoServer** and **UnoLauncher**, which together form the foundation of the application's runtime environment and communication layer.

UnoServer is the backend service responsible for running the core UNO game logic independently of the user interface. Instead of the game logic being tied directly to a local console application, UnoServer exposes the game as a network-accessible service. This design allows the frontend (built in p5.js) to communicate with the backend through HTTP requests, enabling a clean separation between UI and logic. Hosting the game, player, botplayer, rules and deck also starting new games drawing cards retrieving the hands for the players. By centralizing the logic in UnoServer, the team ensures that all gameplay rules are enforced uniformly and that the UI remains lightweight and responsive. UnoLauncher acts as the entry point for running the entire UNO application.

While UnoServer handles the backend logic, UnoLauncher is responsible for starting the backend server and initializing the uno server. UnoLauncher essentially bridges the gap between development and execution. Instead of running multiple Java classes manually, the launcher ensures that the server boots up correctly and consistently every time.

During the integration and testing phase of the Uno application several issues surfaced that required debugging, UI adjustments and time corrections. These challenges were expected as part of merging the backend logic with the interactive frontend, and resolving them significantly improved the overall user experience and system stability. One of the earliest problems involved timing mismatches between UI title cards and game events. Certain title cards such as Player turn and opponent turn would appear out of order sometimes showing before the actual game action occurred. To fix this, the team adjusted the timing sequence in the UI layer so that title cards appear only after the corresponding backend event is fully processed. This ensured that animations card updates and turn indicators were synchronized correctly with the game state. Another issue that occurred when the payer drew many cards. The hand would eventually extend

the past screen boundaries, making some cards inaccessible. This was especially problematic when the player was forced to draw multiple cards due to draw two or wild draw four. The solution was to implement a dynamic card scaling, as the number of cards in the player's hand increases each card automatically becomes smaller allowing the entire hand to remain visible on the screen. This made the UI more flexible and prevented layout breakage.

Initially the opponent's turn happened almost instantly because the bot logic executes extremely fast. The player could not see which card the bot played or when the bot drew a card or skipped a turn. It made the game feel confusing to address this, the team added a floating animation where the bot's card rises before being placed on the discard pile and added a short delay before the bot plays. We decided to add a status message such as if the opponent drew a card or the opponent skipped their turn.

Another issue we encountered was similar to the bot; it was not obvious when the player had their turn skipped due to a skip card or draw two. To fix this the team added a dedicated title card that appears whenever the player loses their turn. This ensured consistency between player and bot. Another issue we encountered occurred when a player won or lost the game state would update internally, but the UI did not transition to an end game. This made the game appear frozen or incomplete.

The team resolved this by implementing a win animation and a lose animation. The team adjusted the timing so that the opponent's final action completes fully before the end game screen. The team performed additional fine tuning to make sure the animation did not overlap, title cards appeared in correct order, the discard pile updates at the right moment and lastly make the bot's action feel more natural rather than instantaneous. This resulted in a smoother and more polished gameplay.

Strengths and Weaknesses:

The application demonstrates several notable strengths. The separation of the game logic, server layer, and user interface into distinct components makes the codebase modular and easy to maintain, in the sense that changes to the UI do not affect the game engine, and vice versa. The REST API design means the interface could theoretically be swapped out for a different frontend entirely without touching the Java backend, making the architecture highly flexible and extensible. The use of p5.js allowed for rich animations and visual feedback, including card preview animations, a turn timer, win and lose screens, and opponent notifications, all of which would have been significantly harder to achieve with a standard Java UI framework. The application also includes a start screen that verifies server connectivity before allowing the

player to begin, providing a polished user experience from the moment the application is launched. Additionally, the Bot opponent provides a functional single-player experience with basic decision-making logic such as color preference and card prioritization, and the game correctly enforces standard UNO rules including special card effects, draw mechanics, and turn order. The multithreaded server design ensures that the HTTP server remains responsive even under rapid or concurrent requests, and the static file serving built into the server means the application requires no external web server to run.

However, the application also has notable weaknesses. While the application was designed to run as a desktop program using JavaFX's WebView component, it was observed that the user interface renders significantly better when accessed through a standard web browser such as Google Chrome. JavaFX's WebView is based on an older embedded browser engine that lacks full support for certain rendering features, resulting in visual inconsistencies such as emoji displaying as boxes and certain stroke-based graphics not rendering correctly. In contrast, Google Chrome uses a modern, fully up-to-date browser engine that supports the complete range of HTML5, CSS3, and JavaScript features used by the p5.js interface, making it the recommended way to interact with the application. Beyond rendering limitations, the server is limited to a single active game at a time, meaning it cannot support multiple concurrent sessions without significant reworking. The Bot opponent, while functional, follows a relatively simple strategy and can be predictable once a player understands its behavior. Finally, the application currently lacks persistent data such as score tracking across multiple rounds, user accounts, or game history, which limits its long-term replayability.

Additional Features and Final Thoughts:

After perfecting the game play, the game lacked something. When the game first boots up, it starts on the game play screen with the game already started. This made the game less appealing to some users who had tested the game, in order to fix this, we integrated a “Start Game” screen or a title card. This meant that when the connection with the server is first established, the title card appears first, instead of the player being thrust into the game from the get go. This move made the Application look a lot more well rounded and made for a better user experience.

In the end the most challenging part proved to be after the game was already made. It was the constant playing and tweaking, then again playing and tweaking. This was also the most important part, in order to ensure that the UNO Game was played as smoothly as possible. There are many things that we could've expanded in the App if not for lack of time, for example, when the player starts a new game (whether it be after pressing the start game button or after pressing play again) the game starts with a shuffling card animation, to further enhance the user's experience, making it more realistic and immersive. Another feature we can expand on is, making the game two-player, but not on a local server, so other human players can also join the game. This would of course need many more months of work and would unfortunately not be possible for a group of college students.